Annotated Bibliography: Trust but Verify - The Evolution of Compiler Validation

Introduction and Motivation

Modern software engineering is built on an implicit foundation of trust: developers trust that the compiler will faithfully translate their high-level source code into semantically equivalent machine code. However, compilers are exceptionally complex software systems, featuring millions of lines of code and aggressive optimization passes. When a compiler contains a bug, it silently introduces errors into otherwise correct programs, rendering traditional application-level debugging highly ineffective. As critical systems such as operating system kernels or cryptographic libraries increasingly rely on aggressive compilation and Just-In-Time (JIT) translation, ensuring the functional correctness of the compiler infrastructure has become a paramount security and reliability concern.

The Evolution of Compiler Verification and Validation

Historically, the programming languages community sought to solve this problem by formally verifying the compiler itself - proving mathematically that the compiler's source code would always generate correct target code. As surveyed by Dave (2003), these early efforts often struggled to scale beyond toy languages or required abandoning legacy infrastructure entirely.

Recognizing the impracticality of verifying massive, evolving codebases like GCC or LLVM, a paradigm shift occurred. Pnueli, Siegel, and Singerman (1998) introduced the concept of Translation Validation. Rather than proving the compiler is universally correct, translation validation assumes the compiler might be flawed and instead verifies each individual compilation. It does this by automatically generating a formal proof that the specific target code produced is semantically equivalent to the original source code. This approach decoupled the verification engine from the compiler's internal heuristics. Necula (2000) demonstrated the practical viability of this approach by successfully applying translation validation to an industry-standard, heavily optimizing compiler (GNU C), proving that rigorous formal methods could handle the messy realities of production-grade optimizations.

Despite these theoretical and practical advances, the broader software industry largely continued to rely on traditional testing methods for compilers. The danger of this complacency was exposed by Yang et al. (2011) with the introduction of CSmith, a randomized test-case generator. By fuzzing mature compilers like GCC and LLVM, the researchers uncovered hundreds of previously unknown, silent miscompilation bugs. This served as a massive reality check for the field, proving that unverified

optimizers were inherently untrustworthy and injecting renewed urgency into compiler correctness research.

In response to the pervasive bugs found in traditional, destructive optimization passes, researchers began exploring novel architectural paradigms for compilation. Traditional compilers apply optimizations in a strict sequence, which introduces the "phase-ordering problem" which is a fundamental dilemma where the order of passes dictates the outcome, because one optimization might inadvertently destroy the code patterns needed by a subsequent pass. Tate et al. (2009) proposed Equality Saturation to bypass this issue entirely. Rather than destructively updating the code step-by-step, their technique simultaneously represents a program in multiple equivalent forms. This structural shift not only resolved the phase-ordering problem but also significantly reduced the surface area for optimization bugs.

More recently, the focus of compiler correctness has expanded from ahead-of-time (AOT) user-space compilers to highly privileged, dynamic compilation environments. Wang et al. (2014) addressed this with Jitk, pushing formal verification into the operating system kernel. Because a miscompilation in an in-kernel JIT (such as those used for eBPF packet filtering) can lead to total system compromise, Wang et al. mechanized the semantics of the JIT infrastructure to prove its correctness. This work represents the synthesis of decades of PL theory applied directly to a critical, low-level systems execution environment.

Open Research Questions

While translation validation and verified compilation have made significant strides, several open questions remain. First, scaling translation validation to handle aggressive, whole-program Link-Time Optimizations (LTO) remains computationally challenging. Second, as compilers increasingly integrate machine learning heuristics to guide optimization passes, verifying the output of these non-deterministic, AI-driven transformations is an open frontier. Finally, minimizing the runtime and memory overhead of verified in-kernel JITs is critical for their widespread adoption in high-performance production environments.

1. Dave, M. A. (2003). Compiler Verification: A Bibliography

- **Problem:** Before modern digital libraries and search engines made traversing citation graphs trivial, researchers entering the niche field of compiler verification lacked a centralized index of prior work. This made it difficult to track the lineage of the field back to its foundational papers.
- **Solution:** The author compiles a raw, chronological bibliography of exactly 100 papers related to compiler verification, mapping the literature from McCarthy and Painter's initial 1967 work up through early 2003.
- **Evaluation:** As a pure bibliography, the document contains no experimental evaluation, comparative analysis, or qualitative assessments of the cited works. It serves strictly as a historical index.
- **Limitations and Future Work:** The primary limitation is the complete lack of synthesis, narrative, or taxonomy; it simply lists titles and authors without summarizing their contributions or categorizing the different verification techniques. Because it is purely an index, it offers no discussion of open problems or future work in the field.

2. Pnueli, A., Siegel, M., & Singerman, E. (1998). Translation validation

- **Problem:** Traditional compiler verification (proving in advance that a compiler will always produce correct target code) is an extremely complex task. Tying a mathematical proof directly to the compiler's internal architecture effectively "freezes" the compiler's design, discouraging future improvements or minor revisions, since any change requires redoing the entire proof.
- **Solution:** The authors introduce the concept of "translation validation." Instead of proving the compiler is universally correct, an Analyzer runs after every individual compilation. It uses a common semantic framework called Synchronous Transition Systems (STS) to represent both the source and target code. It then automatically checks if the target code is a "correct implementation" (a refinement) of the source code, generating a proof script if successful.
- **Evaluation:** The authors evaluate their theoretical framework by applying it to a highly challenging compilation scenario: translating a synchronous, multi-clock data-flow language

(SIGNAL) into asynchronous, sequential C-code. Using the TLV proof system, they successfully verified the refinement rules for a sample SIGNAL program (MUX).

- **Limitations and Future Work:** The authors acknowledge that fully automating the generation of the invariants and refinement mappings relies on the highly predictable, specific structure of the C-code generated by the SACRES compiler (specifically, the repeated execution of a single loop body). Future work requires tailoring this translation validation "tool-set" to handle different translators and code generators for other languages like Esterel or Statecharts.

3. Necula, G. C. (2000). Translation validation for an optimizing compiler

- **Problem:** While Pnueli introduced translation validation, previous applications were restricted to non-optimizing translations or required the compiler to output specific, restricted patterns. The challenge was scaling this technique to a realistic, heavily optimizing, production-grade compiler (the GNU C Compiler, GCC) without needing to modify the compiler itself to emit proofs or simulation relations.
- **Solution:** Necula designed a Translation Validation Infrastructure (TVI) that passively observes GCC's intermediate language (RTL) before and after each compilation pass. The core innovation is the use of symbolic evaluation, which abstracts away minor syntactic differences caused by optimizations like instruction scheduling or register allocation. The TVI uses a two-step inference algorithm: a heuristic-based Branch module matches the control-flow graphs, and a Solve module uses algebraic and aliasing rules to verify that the symbolic states produce equivalent observable behaviors (sequences of function calls and returns).
- **Evaluation:** The TVI was evaluated on massive, real-world C codebases: the GCC compiler itself (approx. 5,900 functions) and the Linux 2.2 kernel (approx. 2,600 functions). The infrastructure successfully handled most intraprocedural optimizations with a manageable 4x overhead to compilation time, and it successfully isolated a known bug in an older version of GCC (2.7.2.2).
- **Limitations and Future Work:** The system's primary limitation is the occurrence of "false alarms" (reporting a semantic mismatch when the code is actually correct). This happens most frequently during loop unrolling because the heuristic Branch module struggles to map the modified control-flow graph. Furthermore, the system does not handle interprocedural optimizations, the reordering of memory operations across function calls, or exception handling. Future work aims to optimize the constraint solver via memoization to improve speed, extend validation to exception handling, and potentially adapt the tool to verify hardware-specific code generators (like x86 and IA-64).

4. Yang, X., Chen, Y., Eide, E., & Regehr, J. (2011). Finding and understanding bugs in C compilers

- **Problem:** While compiler verification is a noble goal, most production compilers (like GCC and LLVM) rely on testing rather than formal proofs. However, traditional fixed test suites are inadequate, and previous attempts at automated random testing failed because they frequently generated C code containing "undefined" or "unspecified" behaviors. If a generated program's semantics are undefined, it is impossible to use it for differential testing (comparing outputs across different compilers) because any output is technically permissible.
- **Solution:** The authors developed Csmith, a highly expressive randomized test-case generator that uses interleaved static analysis and dynamic checks to ensure every generated C program has a single, strictly defined meaning (avoiding all 191 undefined and 52 unspecified behaviors in C99). This guaranteed semantic consistency allows them to employ randomized differential testing: feeding the same generated program into multiple compilers and using majority voting to detect when a compiler silently miscompiles the code.
- **Evaluation:** The evaluation is staggering. Over three years, Csmith uncovered more than 325 previously unknown bugs across 11 different C compilers, including GCC, LLVM, and commercial tools used for safety-critical systems. Every tested compiler crashed and silently generated wrong code. Notably, when testing the formally verified compiler CompCert, Csmith found a few bugs in its unverified front-end, but was completely unable to find any middle-end optimization bugs, providing strong empirical evidence for the value of formal compiler verification.
- **Limitations and Future Work:** Csmith currently lacks support for certain language features (like floating-point types, strings, dynamic memory allocation, and recursion) and does not guarantee that generated programs will terminate. Furthermore, reducing a massive, randomly generated failing test case into a small, reportable bug is extremely difficult because existing automated reduction tools introduce undefined behaviors. Future work suggests exploring the integration of white-box testing techniques, though the authors note the immense engineering challenges involved.

5. Tate, R., Stepp, M., Tatlock, Z., & Lerner, S. (2009). Equality saturation: a new approach to optimization

- **Problem:** Traditional optimizing compilers suffer from the fundamental "phase-ordering problem." Because optimizations are applied sequentially and destructively modify the program, applying one optimization can irreversibly prevent another, potentially more beneficial optimization from triggering. Furthermore, standard profitability heuristics (such as

deciding whether to inline a function) must make localized, isolated decisions without knowing if the transformation will enable or disable future optimizations.

- **Solution:** The authors propose restructuring the optimizer using a paradigm called "Equality Saturation." Instead of destructively altering the code, optimizations act as equality analyses that continually add equivalence information to a novel, referentially transparent intermediate representation called an Equivalence Program Expression Graph (E-PEG). The engine saturates the E-PEG with axioms until it simultaneously encodes exponentially many optimized versions of the input program. Once saturated, a global profitability heuristic (implemented via a Pseudo-Boolean solver) analyzes the graph to extract the single most optimal execution path.
- **Evaluation:** The authors instantiated this approach in Peggy, a Java bytecode optimizer, and evaluated it against the SpecJVM benchmark. Peggy proved practical in both time and space overhead, successfully saturating 84% of methods without requiring bounds. It naturally discovered complex, unanticipated optimizations (like inter-loop strength reduction) without needing specialized transformation rules. Furthermore, Peggy successfully performed translation validation on the mature Soot compiler, verifying 98% of its compilation runs and uncovering a silent miscompilation bug in the remaining 2%.
- **Limitations and Future Work:** The primary theoretical limitation is that equality saturation does not guarantee termination; to prevent infinite expansion (e.g., from unrolling), the engine must sometimes enforce artificial bounds on the number of processed expressions. Practically, the Pseudo-Boolean solver can be computationally expensive, occasionally timing out on complex graphs. Future work aims to extend the system to generate machine-checkable proofs for the optimizations it performs, potentially allowing the compiler to automatically "learn" and generalize new optimizations on the fly.

6. Wang, X., Lazar, D., Zeldovich, N., Chlipala, A., & Tatlock, Z. (2014). Jitk: A trustworthy in-kernel interpreter infrastructure

- **Problem:** Modern operating systems frequently run user-provided code inside the kernel via interpreters (such as the BSD Packet Filter, or BPF, used for system call filtering in Linux's Seccomp). Because these interpreters run with full kernel privileges, any bug in the interpreter, such as control-flow miscalculations, division-by-zero errors, or out-of-bounds memory accesses, can compromise the entire operating system. The complexity of Just-In-Time (JIT) compilation makes these interpreters particularly vulnerable to subtle bugs.
- **Solution:** The authors present Jitk, a formally verified infrastructure for building in-kernel JIT interpreters. To prevent user errors, they introduce a high-level System Call Policy Language

(SCPL) that compiles down to BPF bytecode. To prevent kernel errors, they implemented a verified BPF JIT using the Coq proof assistant, built on top of the CompCert verified compiler framework. The JIT guarantees functional correctness (the native code preserves the semantics of the BPF filter), guarantees termination (no infinite loops), and guarantees bounded stack usage.

- **Evaluation:** Jitk was integrated into the Linux kernel as a drop-in replacement for its existing Seccomp subsystem. By analyzing historical BPF interpreter vulnerabilities, the authors demonstrated that Jitk's formal proofs mathematically prevent all previously known classes of security bugs in this domain. Performance evaluations showed that the native code generated by Jitk was comparable in size (and often smaller on x86) to existing, unverified in-kernel JITs. Furthermore, the end-to-end performance of running OpenSSH with a Jitk-compiled filter was identical to running a hand-written filter, with only a minor ~20ms overhead during the initial compilation phase.
- **Limitations and Future Work:** Jitk's safety guarantees rely on a trusted computing base (TCB) that includes the Coq proof checker, the OCaml runtime, a small unverified I/O stub, and the formal specifications of BPF and SCPL themselves. Additionally, while Jitk ensures the functional correctness of the generated assembly, it does not currently guarantee protection against advanced exploits like "JIT spraying," though the authors note the framework could be extended to prove the correctness of mitigations for such attacks.

7. Willsey, M., Nandi, C., Wang, Y. R., Flatt, O., Tatlock, Z., & Panchekha, P. (2021). egg: Fast and extensible equality saturation.

- **Problem:** While the original equality saturation paradigm (Tate et al., 2009) avoided the phase-ordering problem, it suffered from severe performance bottlenecks. The underlying data structure (the e-graph, originally borrowed from automated theorem provers) was not specialized for compiler workloads. Maintaining the e-graph's strict congruence invariants after every single rewrite was computationally expensive, and integrating domain-specific program analyses (like constant folding or liveness analysis) required ad-hoc, messy extensions.
- **Solution:** The authors introduced egg, a new open-source library that completely reimagined equality saturation. They implemented a technique called "rebuilding" which defers and amortizes the cost of maintaining e-graph invariants, yielding massive asymptotic speedups. Furthermore, they introduced "e-class analyses," a generalized mathematical mechanism that allows the e-graph to natively and cleanly incorporate domain-specific analyses alongside standard syntactic rewrites without breaking the abstraction.

- **Evaluation:** The authors evaluated the egg framework across diverse domains, demonstrating that it provided asymptotic speedups over previous state-of-the-art e-graph implementations - often running dozens of times faster. Highlighting its versatility, they successfully used egg to synthesize optimal floating-point math, discover intricate register allocation policies, and optimize deep learning computational graphs (like XLA tensors), achieving state-of-the-art results that matched or beat specialized, human-designed optimizers.
- **Limitations and Future Work:** While egg solved the major performance hurdles of equality saturation, seamlessly integrating these non-destructive e-graph passes directly into traditional, destructive-update production compilers (like LLVM) remains an ongoing engineering challenge. Additionally, generating minimal, machine-checkable proofs of these optimized graphs was an open challenge at the time of publication, as extracting minimal proof certificates from e-graphs is NP-complete (a problem that was subsequently addressed by researchers in 2022).

Project Proposal

Team: Scott Brinley (Solo)

Topic: Trust but Verify: The Evolution of Compiler Validation and Equality Saturation

Initial Core Papers:

1. *A Survey of Compiler Verification Techniques* (Dave, 2003) – A bibliography mapping the early landscape and historical context of the verification problem.
2. *Translation Validation for an Optimizing Compiler* (Necula, 2000) – The breakthrough paper scaling translation validation to practical, industry-standard C compilers.
3. *Equality Saturation: A New Approach to Optimization* (Tate et al., 2009) – Introduces the referentially transparent E-PEG data structure to bypass the phase-ordering problem.
4. *egg: Fast and Extensible Equality Saturation* (Willsey et al., 2021) – The modern, highly optimized implementation of equality saturation that resolves historical performance bottlenecks.

Topic Description: Modern software engineering relies heavily on the correctness of compilers, but formally verifying massive, evolving codebases like GCC or LLVM from scratch is often impractical. This project will explore the paradigm shift from attempting to verify entire compiler architectures to "Translation Validation"- verifying the output of each individual compilation run to ensure semantics

are preserved. The research will trace this evolution from foundational validation theories to the discovery of pervasive optimization bugs using tools like CSmith. Furthermore, it will explore novel, non-destructive architectural paradigms like Equality Saturation (e-graphs), culminating in modern implementations that optimize complex systems while bridging the gap between systems engineering and formal programming language theory.

Implementation Plan: For the implementation portion, I plan to explore the mechanics of equality saturation. Utilizing a modern framework like the egg library (or by building a simplified e-graph engine from scratch), I will implement a small set of mathematical rewrite rules (axioms) for a toy language or a subset of an intermediate representation. The goal will be to mechanize a proof-of-concept demonstrating how the equality saturation engine avoids the phase-ordering problem by discovering an optimal instruction sequence that traditional, sequential destructive-update passes would typically miss. I expect to refine the specific target language and rule set as I review the literature.